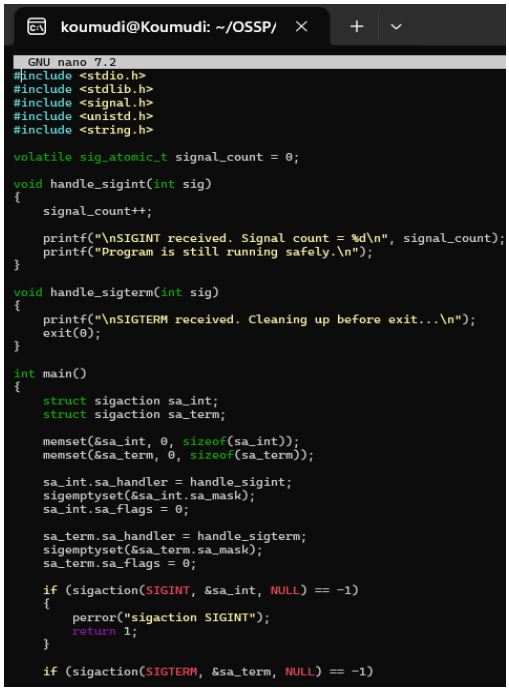
Skill OSSP

WEEK – 10 (S19,20)

SESSION 19:

TASK1: To Register Handlers, Configure Signal Actions, Handle Interrupts, Manage Signal Context, Verify Stability, Test Signal Events.



```
koumudi@Koumudi: ~/OSSP/
 GNU nano 7.2
#include <stdio.h>
#include <stdlib.h>
#include <signal.h>
#include <unistd.h>
#include <string.h>

volatile sig_atomic_t signal_count = 0;

void handle_sigint(int sig)
{
    signal_count++;

    printf("\nSIGINT received. Signal count = %d\n", signal_count);
    printf("Program is still running safely.\n");
}

void handle_sigterm(int sig)
{
    printf("\nSIGTERM received. Cleaning up before exit...\n");
    exit(0);
}

int main()
{
    struct sigaction sa_int;
    struct sigaction sa_term;

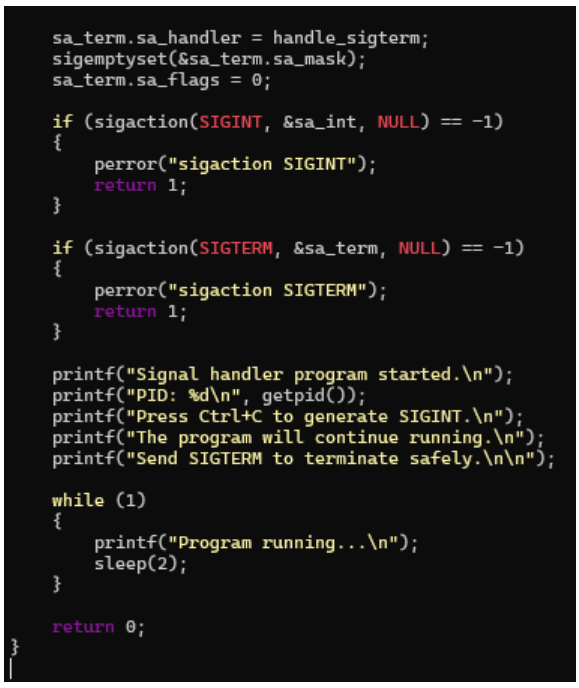
    memset(&sa_int, 0, sizeof(sa_int));
    memset(&sa_term, 0, sizeof(sa_term));

    sa_int.sa_handler = handle_sigint;
    sigemptyset(&sa_int.sa_mask);
    sa_int.sa_flags = 0;

    sa_term.sa_handler = handle_sigterm;
    sigemptyset(&sa_term.sa_mask);
    sa_term.sa_flags = 0;

    if (sigaction(SIGINT, &sa_int, NULL) == -1)
    {
        perror("sigaction SIGINT");
        return 1;
    }

    if (sigaction(SIGTERM, &sa_term, NULL) == -1)
```



```
    sa_term.sa_handler = handle_sigterm;
    sigemptyset(&sa_term.sa_mask);
    sa_term.sa_flags = 0;

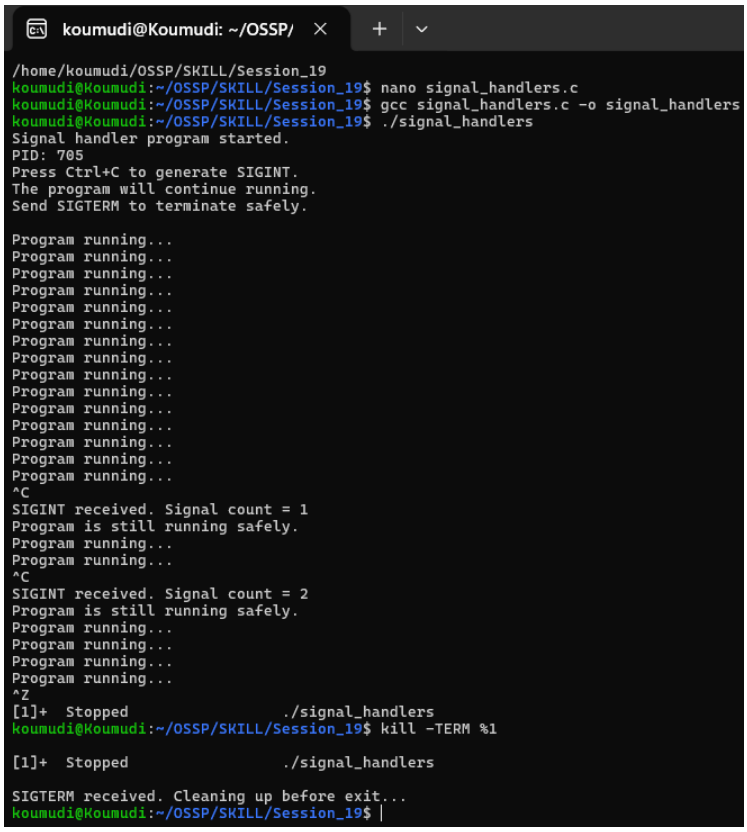
    if (sigaction(SIGINT, &sa_int, NULL) == -1)
    {
        perror("sigaction SIGINT");
        return 1;
    }

    if (sigaction(SIGTERM, &sa_term, NULL) == -1)
    {
        perror("sigaction SIGTERM");
        return 1;
    }

    printf("Signal handler program started.\n");
    printf("PID: %d\n", getpid());
    printf("Press Ctrl+C to generate SIGINT.\n");
    printf("The program will continue running.\n");
    printf("Send SIGTERM to terminate safely.\n\n");

    while (1)
    {
        printf("Program running...\n");
        sleep(2);
    }

    return 0;
}
```



```
koumudi@Koumudi: ~/OSSP/
/home/koumudi/OSSP/SKILL/Session_19
koumudi@Koumudi:~/OSSP/SKILL/Session_19$ nano signal_handlers.c
koumudi@Koumudi:~/OSSP/SKILL/Session_19$ gcc signal_handlers.c -o signal_handlers
koumudi@Koumudi:~/OSSP/SKILL/Session_19$ ./signal_handlers
Signal handler program started.
PID: 705
Press Ctrl+C to generate SIGINT.
The program will continue running.
Send SIGTERM to terminate safely.

Program running...
Program running...
Program running...
Program running...
Program running...
Program running...
Program running...
Program running...
Program running...
Program running...
Program running...
Program running...
Program running...
Program running...
Program running...
Program running...
Program running...
^C
SIGINT received. Signal count = 1
Program is still running safely.
Program running...
Program running...
^C
SIGINT received. Signal count = 2
Program is still running safely.
Program running...
Program running...
Program running...
Program running...
^Z
[1]+  Stopped                 ./signal_handlers
koumudi@Koumudi:~/OSSP/SKILL/Session_19$ kill -TERM %1

[1]+  Stopped                 ./signal_handlers

SIGTERM received. Cleaning up before exit...
koumudi@Koumudi:~/OSSP/SKILL/Session_19$
```

TASK2: To Capture SIGINT, Forward Signals, Protect Shell Process, Terminate Foreground Jobs, Update Job States, Test Interrupt Handling.

koumudi@Koumudi: ~/OSSP/

GNU nano 7.2

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>
#include <sys/wait.h>

pid_t foreground_pid = -1;

void handle_sigint(int sig)
{
    printf("\nShell: SIGINT received.\n");

    if (foreground_pid > 0)
    {
        printf("Shell: Forwarding SIGINT to foreground job PID %d.\n",
               foreground_pid);

        kill(foreground_pid, SIGINT);
    }
}

void child_sigint(int sig)
{
    printf("\nForeground job: SIGINT received. Terminating...\n");
    exit(0);
}

int main()
{
    pid_t pid;
    int status;

    signal(SIGINT, handle_sigint);

    printf("Shell process PID: %d\n", getpid());
    printf("Shell is protected from direct termination.\n");

    pid = fork();

    if (pid < 0)
    {
        perror("fork");
```

koumudi@Koumudi: ~/OSSP/

GNU nano 7.2

```c
    pid = fork();

    if (pid < 0)
    {
        perror("fork");
        return 1;
    }

    if (pid == 0)
    {
        signal(SIGINT, child_sigint);

        printf("Foreground job started. PID = %d\n", getpid());

        while (1)
        {
            printf("Foreground job running...\n");
            sleep(2);
        }
    }

    foreground_pid = pid;

    printf("Shell: Foreground job PID = %d\n", foreground_pid);
    printf("Press Ctrl+C to interrupt the foreground job.\n\n");

    waitpid(pid, &status, 0);

    foreground_pid = -1;

    if (WIFEXITED(status))
    {
        printf("\nShell: Foreground job terminated.\n");
        printf("Shell: Job state updated to Completed.\n");
    }

    printf("Shell: Shell process is still running.\n");

    return 0;
}
```

```
koumudi@Koumudi:~/OSSP/SKILL/Session_19$ nano sigint_forward.c
[1]+  Done                    ./signal_handlers
koumudi@Koumudi:~/OSSP/SKILL/Session_19$ gcc sigint_forward.c -o sigint_forward
koumudi@Koumudi:~/OSSP/SKILL/Session_19$ ./sigint_forward
Shell process PID: 731
Shell is protected from direct termination.
Shell: Foreground job PID = 732
Press Ctrl+C to interrupt the foreground job.

Foreground job started. PID = 732
Foreground job running...
Foreground job running...
^C

Shell: SIGINT received.
Foreground job: SIGINT received. Terminating...
Shell: Forwarding SIGINT to foreground job PID 732.

Shell: Foreground job terminated.
Shell: Job state updated to Completed.
Shell: Shell process is still running.
koumudi@Koumudi:~/OSSP/SKILL/Session_19$
```

SESSION 20:

TASK1: To Capture SIGTSTP, Suspend Processes, Update Job Table, Preserve State, Resume Later, Test Suspension Workflow.

koumudi@Koumudi: ~/OSSP/

GNU nano 7.2

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>
#include <sys/wait.h>
#include <string.h>

typedef struct
{
    int job_id;
    pid_t pid;
    char status[20];
} Job;

Job job;

void handle_sigtstp(int sig)
{
    printf("\nParent: SIGTSTP received.\n");

    if (job.pid > 0)
    {
        printf("Parent: Stopping job [%d], PID = %d\n",
               job.job_id, job.pid);

        kill(job.pid, SIGSTOP);
        strcpy(job.status, "Stopped");

        printf("Parent: Job state updated to %s.\n",
               job.status);
    }
}

int main()
{
    pid_t pid;
    int status;

    job.job_id = 1;
    job.pid = -1;
    strcpy(job.status, "Starting");

    signal(SIGTSTP, handle_sigtstp);

    pid = fork();
```

koumudi@Koumudi: ~/OSSP/

GNU nano 7.2

```c
    sleep(4);

    printf("Parent: Sending SIGSTOP to demonstrate suspension.\n");
    kill(job.pid, SIGSTOP);

    waitpid(job.pid, &status, WUNTRACED);

    if (WIFSTOPPED(status))
    {
        strcpy(job.status, "Stopped");

        printf("Parent: Job [%d] state = %s\n",
               job.job_id, job.status);

        printf("Parent: Preserving job information.\n");

        sleep(2);

        printf("\nParent: Sending SIGCONT to resume job.\n");
        kill(job.pid, SIGCONT);

        strcpy(job.status, "Running");

        printf("Parent: Job [%d] state = %s\n",
               job.job_id, job.status);
    }

    sleep(4);

    printf("\nParent: Terminating job.\n");
    kill(job.pid, SIGTERM);

    waitpid(job.pid, &status, 0);

    strcpy(job.status, "Completed");

    printf("Parent: Job [%d] state = %s\n",
           job.job_id, job.status);

    printf("\nSuspension and resume workflow completed.\n");

    return 0;
}
```

koumudi@Koumudi: ~/OSSP/

```
koumudi@Koumudi:~$ cd ~/OSSP/SKILL/Session_20
-bash: cd: /home/koumudi/OSSP/SKILL/Session_20: No such file or directory
koumudi@Koumudi:~$ mkdir -p ~/OSSP/SKILL/Session_20
cd ~/OSSP/SKILL/Session_20
koumudi@Koumudi:~/OSSP/SKILL/Session_20$ pwd
/home/koumudi/OSSP/SKILL/Session_20
koumudi@Koumudi:~/OSSP/SKILL/Session_20$ nano sigtstp_job.c
koumudi@Koumudi:~/OSSP/SKILL/Session_20$ gcc sigtstp_job.c -o sigtstp_job
koumudi@Koumudi:~/OSSP/SKILL/Session_20$ ./sigtstp_job
Parent: Job [1] created. PID = 782
Parent: Job state = Running
Press Ctrl+Z to suspend the job.
After suspension, the job will be resumed automatically.

Child: Job started. PID = 782
Child: Job is running...
Child: Job is running...
Parent: Sending SIGSTOP to demonstrate suspension.
Parent: Job [1] state = Stopped
Parent: Preserving job information.

Parent: Sending SIGCONT to resume job.
Parent: Job [1] state = Running
Child: Job is running...
Child: Job is running...

Parent: Terminating job.
Parent: Job [1] state = Completed

Suspension and resume workflow completed.
koumudi@Koumudi:~/OSSP/SKILL/Session_20$
```

TASK2: To Create Process Groups, Assign Group IDs, Transfer Terminal Control, Restore Ownership, Coordinate Signals, Validate Behavior.

koumudi@Koumudi: ~/OSSP/

```
GNU nano 7.2
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>
#include <sys/wait.h>

void child_work(const char *name)
{
    printf("%s: PID = %d, PGID = %d\n",
           name, getpid(), getpgrp());

    for (int i = 1; i <= 5; i++)
    {
        printf("%s: Working... %d\n", name, i);
        fflush(stdout);
        sleep(1);
    }

    printf("%s: Completed.\n", name);
    exit(0);
}

int main()
{
    pid_t child1, child2;
    pid_t shell_pgid = getpgrp();
    int status;

    printf("Parent PID  = %d\n", getpid());
    printf("Parent PGID = %d\n", shell_pgid);

    child1 = fork();

    if (child1 < 0)
    {
        perror("fork");
        return 1;
    }

    if (child1 == 0)
    {
        setpgid(0, 0);
        child_work("Child 1");
    }

    setpgid(child1, child1);

    child2 = fork();

    if (child2 < 0)
    {
        perror("fork");
        return 1;
```

koumudi@Koumudi: ~/OSSP/

```
GNU nano 7.2
    if (child2 < 0)
    {
        perror("fork");
        return 1;
    }

    if (child2 == 0)
    {
        setpgid(0, child1);
        child_work("Child 2");
    }

    setpgid(child2, child1);

    printf("\nProcess group created.\n");
    printf("Child 1 PID = %d\n", child1);
    printf("Child 2 PID = %d\n", child2);
    printf("Shared Process Group ID = %d\n", child1);

    printf("\nTransferring terminal control to process group...\n");

    if (tcsetpgrp(STDIN_FILENO, child1) == -1)
    {
        perror("tcsetpgrp");
    }
    else
    {
        printf("Terminal control transferred to process group %d.\n",
               child1);
    }

    printf("\nSending signal to the entire process group...\n");
    kill(-child1, SIGCONT);

    waitpid(child1, &status, 0);
    waitpid(child2, &status, 0);

    printf("\nRestoring terminal control to parent...\n");

    if (tcsetpgrp(STDIN_FILENO, shell_pgid) == -1)
    {
        perror("tcsetpgrp");
    }
    else
    {
        printf("Terminal control restored to parent.\n");
    }

    printf("Process group coordination completed.\n");

    return 0;
}
```

```
koumudi@Koumudi:~/OSSP/SKILL/Session_20$ nano process_groups.c
koumudi@Koumudi:~/OSSP/SKILL/Session_20$ gcc process_groups.c -o process_groups
koumudi@Koumudi:~/OSSP/SKILL/Session_20$ ./process_groups
Parent PID  = 797
Parent PGID = 797
Child 1: PID = 798, PGID = 798
Child 1: Working... 1

Process group created.
Child 1 PID = 798
Child 2 PID = 799
Shared Process Group ID = 798
Child 2: PID = 799, PGID = 798

Child 2: Working... 1
Transferring terminal control to process group...
Terminal control transferred to process group 798.

Sending signal to the entire process group...
Child 1: Working... 2
Child 2: Working... 2
Child 1: Working... 3
Child 2: Working... 3
Child 1: Working... 4
Child 2: Working... 4
Child 2: Working... 5
Child 1: Working... 5
Child 2: Completed.
Child 1: Completed.

Restoring terminal control to parent...

[1]+  Stopped                 ./process_groups
koumudi@Koumudi:~/OSSP/SKILL/Session_20$
```